

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Proyecto: Logic Detective

PONDERACIÓN: 30 pts

 **Tiempo estimado: 50 hrs/min**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor.....	3
1.2. Competencia(s).....	3
1.3. Objetivo SMART.....	3
2. Resumen Ejecutivo.....	4
3. Enunciado del Proyecto.....	4
3.1 Descripción del problema a resolver.....	4
3.2 Alcance del proyecto.....	4
3.3 Entregables.....	5
4. Material de apoyo.....	7
5. Metodología.....	7
6. Recursos y herramientas a utilizar.....	8
7. Desarrollo de Habilidades Blandas.....	8
8. Cronograma.....	9
9. Rúbrica de Calificación.....	10
9.1 Requisitos para optar a la calificación.....	10
9.2 Resumen de Puntuaciones.....	12
9.5 Comentarios Generales.....	15
Detalle de la Calificación.....	15

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Nombre del valor	Pensamiento crítico y perseverancia
¿Cómo se aplica en el laboratorio?	Los estudiantes deberán analizar evidencias, comparar declaraciones, identificar contradicciones y construir conclusiones fundamentadas mediante reglas lógicas. La perseverancia se aplicará al depurar hechos, consultas y predicados en Prolog, revisando los resultados hasta obtener inferencias coherentes y verificables. Además, deberán evitar acusaciones basadas en una sola evidencia y justificar cada conclusión mediante el razonamiento implementado en el sistema.

1.2. Competencia(s)

Tipo de Competencia	
Competencia general del curso	Aplica principios básicos de ingeniería, ciencias de la computación y sistemas de información y comunicación en la formulación y resolución adecuada de problemas complejos.
Competencia específica del curso	Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación, negación y cortes en Prolog para modelar bases de conocimiento y resolver problemas de inferencia lógica.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de diseñar e implementar un sistema experto de investigación denominado Logic Detective, en el que representarán sospechosos, evidencias, declaraciones, coartadas, ubicaciones y relaciones mediante hechos y reglas de Prolog.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El sistema deberá incluir al menos tres casos de investigación, cuatro sospechosos por caso, diez evidencias, diez reglas de inferencia y un conjunto mínimo de diez casos de prueba, resolviendo correctamente al menos el 80 % de las consultas evaluadas.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	El proyecto será desarrollado utilizando SWI-Prolog, Python y PySwip, aplicando los contenidos trabajados en clase: hechos, reglas, consultas, variables, predicados, listas, recursividad, unificación, negación y cortes.

Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	El proyecto permitirá fortalecer el pensamiento lógico, el análisis de información, la detección de contradicciones y la toma de decisiones fundamentadas, competencias necesarias para desarrollar posteriormente sistemas inteligentes más complejos.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	El sistema deberá completarse en un plazo máximo de cuatro semanas, realizando entregas progresivas de análisis, base de conocimiento, motor de inferencia, integración, pruebas y documentación.

2. Resumen Ejecutivo

El proyecto Logic Detective consiste en el diseño e implementación de un sistema experto basado en lógica computacional e inferencia utilizando Prolog como motor principal de razonamiento, integrado con una aplicación desarrollada en Python. El problema que se busca resolver es la dificultad para analizar múltiples evidencias, declaraciones y relaciones entre sospechosos de manera objetiva durante una investigación, simulando el razonamiento utilizado por un detective. Como solución, se desarrollará un sistema capaz de representar hechos, reglas y relaciones lógicas para analizar casos de investigación, identificar contradicciones, validar coartadas, determinar niveles de sospecha y justificar las conclusiones alcanzadas mediante inferencia lógica. A través de este proyecto, los estudiantes fortalecerán competencias en programación lógica, modelado de bases de conocimiento, desarrollo de sistemas expertos, integración entre Python y Prolog, resolución de problemas complejos y documentación técnica, aplicando conceptos como hechos, reglas, listas, recursividad, unificación, negación y cortes para la construcción de soluciones inteligentes.

3. Enunciado del Proyecto

El proyecto **Logic Detective** tiene como finalidad desarrollar un sistema experto capaz de analizar casos de investigación mediante técnicas de lógica e inferencia. El sistema permitirá representar personas, evidencias, declaraciones, relaciones, lugares, horarios y demás elementos propios de una investigación criminal o de un incidente, utilizando una base de conocimiento implementada en **Prolog**.

A partir de la información registrada, el sistema realizará inferencias para identificar posibles sospechosos, validar o descartar coartadas, detectar contradicciones entre testimonios y evidencias, establecer niveles de sospecha y justificar las conclusiones obtenidas mediante reglas lógicas. El proyecto busca simular el razonamiento de un investigador, demostrando cómo la inteligencia artificial simbólica puede emplearse para apoyar la toma de decisiones basada en conocimiento.

3.1 Descripción del problema a resolver

En una investigación es común encontrar una gran cantidad de información proveniente de diferentes fuentes, como declaraciones de testigos, evidencias físicas, registros de acceso, cámaras de seguridad, horarios y relaciones entre las personas involucradas. Analizar toda esta información de forma manual puede ser un proceso lento, propenso a errores y susceptible a interpretaciones subjetivas.

El objetivo de este proyecto es diseñar e implementar un **sistema experto denominado Logic Detective**, capaz de representar una investigación mediante una base de conocimiento en **Prolog**, donde cada hecho y regla contribuya al razonamiento lógico del sistema. A partir de las evidencias ingresadas, el motor de inferencia deberá determinar qué sospechosos tuvieron la oportunidad, el motivo y los medios para cometer el incidente, identificar inconsistencias entre declaraciones y evidencias, validar coartadas y generar una conclusión fundamentada que explique el razonamiento seguido para resolver el caso.

De esta manera, el proyecto permitirá aplicar técnicas de inteligencia artificial simbólica para modelar conocimiento, realizar inferencias y apoyar la toma de decisiones, simulando el comportamiento de un sistema experto utilizado en procesos de investigación.

3.2 Alcance del proyecto

El sistema inteligente **Logic Detective** estará compuesto por tres componentes principales: un **módulo de investigación**, destinado al usuario que resolverá los casos; un **módulo administrativo**, encargado de gestionar la información de cada investigación; y un **motor de inferencia desarrollado en Prolog**, responsable de analizar hechos, aplicar reglas y generar conclusiones lógicas.

La aplicación deberá desarrollarse bajo una arquitectura web, contenerizar mediante **Docker y Docker Compose**, incluir pruebas automatizadas, incorporar un proceso básico de **CI/CD** y desplegarse en una máquina virtual en la nube utilizando **AWS o Google Cloud Platform**.

La lógica relacionada con la identificación de sospechosos, validación de coartadas, detección de contradicciones y determinación del responsable deberá implementarse en Prolog. Python se utilizará para la API, la integración con Prolog, la administración de datos y la comunicación con la interfaz.

Inicio

Al ingresar a la aplicación, el usuario visualizará una pantalla principal con:

- Nombre y descripción de **Logic Detective**.
- Propósito del sistema.
- Casos de investigación disponibles.
- Opción para iniciar una investigación.
- Opción para acceder al módulo administrativo.
- Estado de los casos iniciados o completados.

El usuario podrá seleccionar un caso disponible e iniciar el proceso de investigación. Los casos podrán presentarse de manera directa o clasificarse según su nivel de dificultad.

Módulo de Investigación

Este módulo permitirá al usuario asumir el rol de detective y resolver casos mediante la consulta de personas, lugares, evidencias, testimonios y acontecimientos.

El usuario no recibirá toda la información desde el inicio. Deberá realizar acciones de investigación para descubrir progresivamente los elementos del caso.

Como mínimo, el sistema deberá permitir:

- Seleccionar un caso de investigación.
- Visualizar la descripción inicial del incidente.
- Consultar la lista de sospechosos.
- Interrogar a sospechosos y testigos.
- Investigar lugares relacionados con el caso.
- Examinar evidencias disponibles.
- Consultar relaciones entre las personas involucradas.
- Analizar motivos y oportunidades.
- Revisar coartadas.
- Consultar la línea temporal de los acontecimientos.
- Detectar posibles contradicciones.
- Solicitar una pista al sistema.
- Consultar el nivel de sospecha de cada persona.
- Emitir una acusación final.
- Visualizar el resultado de la investigación.
- Consultar la explicación lógica generada por el sistema.

Cada acción realizada por el usuario deberá quedar registrada en una bitácora de investigación.

Módulo Administrativo

El motor de inferencia será el componente principal de inteligencia artificial del proyecto. Deberá utilizar una base de conocimiento formada por hechos, reglas, predicados y relaciones lógicas.

Toda deducción relacionada con la resolución de los casos deberá ejecutarse en Prolog y no podrá sustituirse por condiciones programadas directamente en Python.

El motor lógico deberá ser capaz de inferir, como mínimo:

- Personas que tuvieron acceso al lugar.
- Personas que tuvieron oportunidad de cometer el incidente.
- Personas que poseen un posible motivo.
- Personas que contaban con los medios necesarios.

- Personas con una coartada válida.
- Personas con una coartada inválida.
- Declaraciones que contradicen otras declaraciones.
- Declaraciones que contradicen una evidencia.
- Sospechosos que proporcionaron información falsa.
- Evidencias relacionadas con cada sospechoso.
- Relaciones relevantes entre las personas.
- Nivel de sospecha de cada involucrado.
- Posibles cómplices.
- Principal sospechoso.
- Responsable lógico del caso.
- Explicación de las reglas utilizadas para obtener la conclusión.

La base de conocimiento deberá demostrar el uso de:

- Hechos.
- Reglas.
- Consultas.
- Variables.
- Predicados.
- Listas.
- Recursividad.
- Unificación.
- Negación.
- Cortes.

Alcance obligatorio

El proyecto deberá cumplir, como mínimo, con los siguientes requerimientos:

- Desarrollar una interfaz funcional en Python (escritorio o web).
- Implementar toda la lógica de inferencia utilizando Prolog.
- Modelar al menos **tres casos de investigación**.
- Cada caso deberá incluir como mínimo:
 - 4 sospechosos.
 - 10 evidencias.
 - 5 lugares.
 - 5 declaraciones.
 - 10 reglas de inferencia.
- Implementar consultas relacionadas con sospechosos, evidencias, coartadas, motivos y conclusiones.
- Detectar contradicciones entre evidencias y declaraciones.
- Justificar cada conclusión indicando las reglas activadas.
- Generar un informe final del caso.

Alcance opcional

Como funcionalidades adicionales, el estudiante podrá implementar una o más de las siguientes características:

- Selección aleatoria del caso al iniciar la aplicación.
- Sistema de puntuación basado en la cantidad de consultas realizadas.
- Temporizador para resolver el caso.
- Niveles de dificultad.
- Sistema de pistas.
- Visualización gráfica de la relación entre sospechosos y evidencias.
- Exportación del informe en formato PDF.
- Historial de investigaciones resueltas.
- Motor para generar nuevos casos a partir de archivos JSON o CSV.
- Modo multicaso con estadísticas de resolución.

3.3 Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar el proyecto. A continuación, se muestran ejemplos de entregables que pueden adaptarse según el tipo de proyecto asignado:

- Para un proyecto de desarrollo, suelen incluirse elementos como prototipos funcionales, código fuente, pruebas y manuales de usuario.
- Para un proyecto de documentación o investigación, suelen incluirse informes técnicos, estados del arte, propuestas metodológicas y presentaciones de resultados.

Estas tablas sirven como referencia y guía de ejemplos; cada tutor podrá modificarlas, ampliar o eliminar elementos según los objetivos y alcances del proyecto.

Ejemplo para proyectos de desarrollo:

Tipo	Descripción
Código Fuente	Repositorio en GitHub que incluya la interfaz web/escritorio en Python, la base de conocimiento estructurada en Prolog y los scripts de pruebas automatizadas.
Archivos de Despliegue	Archivos <code>Dockerfile</code> y <code>docker-compose.yml</code> funcionales, junto con la configuración del pipeline de CI/CD.

Manual de Usuario	Documento que explica cómo usar el sistema desarrollado, incluyendo capturas de pantalla, pasos detallados y resolución de problemas comunes.
Documentación Técnica	Incluye manuales de usuario, guías de instalación, diagramas y descripciones de la arquitectura o flujo del sistema desarrollado.
Informe de distribución del trabajo grupal	El coordinador del grupo deberá presentar un informe en el que detalle las actividades realizadas por cada integrante y asigne el porcentaje de participación correspondiente. La suma de los porcentajes deberá ser igual al 100 %. El coordinador será responsable de organizar al equipo, verificar el cumplimiento de las tareas, consolidar los entregables y comunicar cualquier incumplimiento o inconveniente ocurrido durante el desarrollo del proyecto.
Diagrama de Flujo	Representación gráfica del flujo de trabajo o funcionamiento del sistema, que permite comprender el recorrido de los datos o acciones dentro del proyecto.

4. Material de apoyo

- **Documentación oficial de SWI-Prolog:** <https://www.swi-prolog.org/>
- **Guía de PySwip (Integración Python-Prolog):** <https://pypi.org/project/pyswip/>
- **Documentación oficial de Docker y Docker Compose:** <https://docs.docker.com/>
- **Introducción a GitHub Actions (CI/CD):** <https://docs.github.com/es/actions>

5. Metodología

La metodología describe el proceso que los estudiantes deben seguir para llevar a cabo el proyecto. En esta sección se ofrecen ejemplos de enfoques y fases de trabajo que pueden adaptarse según el tipo de proyecto (desarrollo o documentación), el curso y las competencias que se desean reforzar. La idea es que los estudiantes comprendan no solo qué deben entregar, sino también cómo llegar a ese resultado. El tutor podrá seleccionar, combinar o ajustar las fases y actividades de acuerdo con los recursos disponibles y los objetivos del curso.

Ejemplo Proyecto de Desarrollo:

1. **Investigación preliminar:** Los estudiantes deben realizar una investigación sobre las tecnologías de automatización y el control de dispositivos mediante aplicaciones móviles.
2. **Diseño del sistema:** Crear diagramas de flujo o esquemas de la aplicación que representen cómo los dispositivos serán controlados.
3. **Desarrollo:** Los estudiantes deberán implementar su solución en fases, asegurándose de cumplir con los requerimientos técnicos descritos.
4. **Pruebas y ajustes:** Se realizarán pruebas para verificar que la aplicación funciona correctamente en diferentes condiciones. Se documentan las pruebas y sus resultados.

Ejemplo Proyecto de Documentación / Investigación:

1. **Planteamiento del problema:** Los estudiantes deben definir de forma clara el tema o pregunta de investigación..
2. **Revisión bibliográfica:** El estudiante debe realizar la selección de fuentes confiables (libros, papers, artículos).
3. **Desarrollo del análisis:** Aplicación de la metodología propuesta (ej. matriz comparativa, simulación conceptual, análisis estadístico). Sistematización de la información recolectada.
4. **Resultados y conclusiones:** Presentación clara de hallazgos en tablas o gráficas.

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- Software: Python, Prolog, Pyswip.
- Plataformas: UEDI, GitHub.
- Lecturas recomendadas: manuales, artículos, documentación oficial.

7. Desarrollo de Habilidades Blandas

- **Pensamiento crítico y analítico:** Capacidad para analizar múltiples declaraciones y evidencias, identificando inconsistencias para trasladarlas a reglas lógicas.
- **Perseverancia y resolución de problemas:** Depuración iterativa de hechos, reglas, recursividad y cortes en Prolog hasta obtener inferencias coherentes y verificables.
- **Trabajo en equipo (si aplica):** Asignación de roles claros (desarrollo frontend/backend, motor lógico, despliegue) y resolución de conflictos técnicos mediante el consenso.
- **Autogestión del tiempo:** Planificación adecuada para cumplir con el cronograma del proyecto y las entregas de integración continua.

8. Cronograma

El cronograma describe las etapas clave del proyecto, los plazos estimados para cada una, y el proceso de asignación, elaboración y calificación de las tareas. Los estudiantes deberán seguir este plan para asegurar que el proyecto avance de manera organizada y cumpla con

los plazos establecidos. Cada fase incluye la asignación de tareas, el tiempo estimado para su elaboración, y el momento de su calificación.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	31/07/2026	31/07/2026
Elaboración	31/07/2026	28/08/2026
Calificación	29/08/2026	29/08/2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Documentación	Manual técnico, manual de usuario y diagramas arquitectónicos redactados con claridad.	
Gestión de Repositorio y CI/CD	Uso correcto de Git, commits descriptivos, integración continua básica configurada y funcional.	
Motor de Inferencia en Prolog	Modelado correcto de los 3 casos mínimos (sospechosos, evidencias, declaraciones). Uso comprobable de listas, recursividad, unificación, negación y cortes.	
Desarrollo e Integración en Python	Interfaz funcional (módulos de investigación y administración), comunicación exitosa con Prolog (PySwip) y manejo correcto del flujo de la aplicación.	
Contenedorización y Despliegue en la Nube	Sistema correctamente contenedorizado (Docker/Compose) y accesible desde una máquina virtual en la nube (AWS/GCP).	

Informe de participación grupal	El coordinador deberá entregar la distribución de actividades y el porcentaje de participación de cada integrante. Los porcentajes deben sumar 100 % y estar respaldados por los commits, documentos, tareas o evidencias del proyecto.	
---------------------------------	---	--

Responsabilidades del coordinador del grupo

El coordinador de cada grupo es responsable de:

- Organizar al equipo y distribuir las actividades de manera equitativa.
- Definir responsables y fechas internas para cada tarea.
- Dar seguimiento al cumplimiento de las actividades asignadas.
- Verificar que los integrantes participen en el desarrollo, documentación, pruebas y presentación del proyecto.
- Mantener comunicación con todos los integrantes del grupo.
- Informar oportunamente al tutor sobre incumplimientos, falta de participación, abandono del grupo o conflictos que no puedan resolverse internamente.
- Verificar que el repositorio contenga los aportes de los integrantes mediante commits identificables.
- Revisar que los entregables estén completos antes de realizar la entrega final.
- Presentar la distribución de actividades y el porcentaje de participación de cada integrante.
- Adjuntar evidencias que respalden los porcentajes asignados, como commits, documentos, tareas realizadas, bitácoras o registros de reuniones.

La suma de los porcentajes de participación deberá ser igual al 100 %. El coordinador deberá procurar que la distribución reportada sea objetiva y esté respaldada por evidencias verificables.

GIT SECCIÓN A:

Tutor 1: roberto1206

Tutor 2: JavierB20

9.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades	40	
Documentación	10	
Gestión de Repositorio y CI/CD	30	
Sub-Total Habilidades		
2. Conocimiento	60	
Motor de Inferencia en Prolog	20	
Desarrollo e Integración en Python	20	
Contenedorización y Despliegue en la Nube (GCP, AWS, Docker)	20	
TOTAL	100	

*La calificación debe incluir ambas áreas conocimientos y habilidades.

9.5 Comentarios Generales

Detalle de la Calificación

Criterio de Evaluación	Punteo Máx	Satisfactorio (100% - 61%)	Necesita Mejorar (60% - 0%)	Rango Satisfactorio	Rango Necesita Mejorar
1. Habilidades	40				
1.1 Documentación	10	Manual técnico, manual de usuario y diagramas arquitectónicos redactados con total claridad.	Documentación incompleta, confusa, superficial o ausente.	10 - 6	5 - 0
1.2 Gestión de Repositorio y CI/CD	30	Uso correcto de Git, commits descriptivos, integración continua básica configurada y funcional.	Uso deficiente de Git, falta de commits descriptivos o el pipeline CI/CD falla/no existe.	30 - 18	17 - 0
2. Conocimientos	60				
2.1 Motor de Inferencia en Prolog	20	Modelado correcto de 3 casos mínimos; uso comprobable de listas, recursividad, unificación, negación y cortes.	No cumple con los 3 casos mínimos o no implementa la lógica deductiva esperada en Prolog.	20 - 12	11 - 0
2.2 Desarrollo e Integración en Python	20	Interfaz funcional (módulos investigación/administración) y comunicación exitosa con Prolog vía PySwip.	La interfaz presenta bloqueos, errores lógicos o no logra comunicarse correctamente con Prolog.	20 - 12	11 - 0
2.3 Contenedorización y Despliegue	20	Sistema correctamente contenedorizado (Docker/Compose) y accesible desde máquina virtual en AWS/GCP.	Contenedores con errores de configuración o el sistema no se encuentra desplegado/accesible en la nube.	20 - 12	11 - 0

